Practical Insights into Data Analysis and Machine Learning

PROGRAM - 5

Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.

Objective

To implement the Locally Weighted Regression (LWR) algorithm and demonstrate its effectiveness in fitting data points.

5. Introduction

The Locally Weighted Regression (LWR) algorithm, a non-parametric regression technique that fits data points by assigning weights to training instances based on their distance from the query point. Unlike traditional linear regression, which finds a single global model, LWR builds a localized model for each prediction, making it highly effective for capturing non-linear patterns in data.

The key idea behind LWR is to apply a weighted least squares approach, where closer points have a greater influence on the regression model than farther points. This is achieved using a kernel function, typically the Gaussian kernel, to assign exponentially decaying weights based on distance. In this experiment, we select an appropriate dataset or generate the synthetic dataset and apply LWR to fit the data points. We visualize the results by plotting the learned regression curve along with the original data points. Through this, we aim to demonstrate how LWR can effectively model complex relationships without assuming a fixed functional form.

5.1 Locally Weighted Regression (LWR)

- Locally Weighted Regression (LWR), also known as Locally Weighted Scatterplot Smoothing (LOWESS or LOESS), is a non-parametric regression algorithm that fits a model to data points in a localized region rather than assuming a single global function for the entire dataset. This makes it highly effective for modeling non-linear relationships in data.
- Each training instance is assigned a weight based on its distance from the query point.
- The Gaussian kernel function is commonly used for computing weights

$$w_i = \exp\left(-\frac{(x_i - x_q)^2}{2\tau^2}\right)$$

where,

- x_q is the query point and τ is the bandwidth parameter that controls how much influence distant points have.
- A small τ results in a highly flexible model but may lead to overfitting.
- A large τ results in a smoother model but may fail to capture local variations.

Using the weighted dataset, LWR applies least squares regression to determine the best-fit line (or curve) for the localized data.

Steps in LWR Algorithm

1. Take input and choose a Query Point x_q

Given a dataset (X, Y) where:

- $X = \{x_1, x_2, ..., x_n\}$ are the input feature values.
- $Y = \{y_1, y_2, ..., y_n\}$ are the corresponding target values.

Select a query point x_q where we want to estimate the target value $\hat{y}_q$.

Choose the bandwidth parameter (tau): This parameter controls the influence of nearby data points.

2. Weight Calculation: For each data point in the training dataset (X):

- Each training point x_i is assigned a weight w_i based on its distance from the query point x_q.
- Apply a kernel function (Gaussian) to the distance to determine the weight.

$$w_i = \exp\left(-\frac{(x_i - x_q)^2}{2\tau^2}\right)$$

- The kernel function assigns higher weights to data points closer to the query point and lower weights to points farther away.
- The bandwidth parameter (tau) controls the width of the kernel, influencing how quickly the weights decrease with distance.

3. Weighted Least Squares: Perform a weighted least squares regression

- Create a diagonal weight matrix (W) using the calculated weights.
- Form a weighted design matrix (X_w) by multiplying the original input features (X) with the square root of the weights.
- Form a weighted target vector (y_w) by multiplying the original target values (y) with the square root of the weights.

$$X' = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix} \qquad W = \begin{bmatrix} w_1 & 0 & \dots & 0 \\ 0 & w_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & w_n \end{bmatrix}$$

- Calculate the regression coefficients (beta) using the formula:

$$\theta = (X'^T W X')^{-1} X'^T W Y$$

where:

X' is the design matrix.
W is the weight matrix.
Y is the target values vector.

- This equation minimizes the weighted sum of squared errors:

$$\sum_{i=1}^{n} w_i(y_i - (\theta_0 + \theta_1 x_i))^2$$

giving more importance to points closer to x_q

4. Prediction:

- Predict y_q for the Query Point. Using the estimated parameters θ, compute the predicted value:

$$\hat{y_q} = \theta_0 + \theta_1 x_q$$

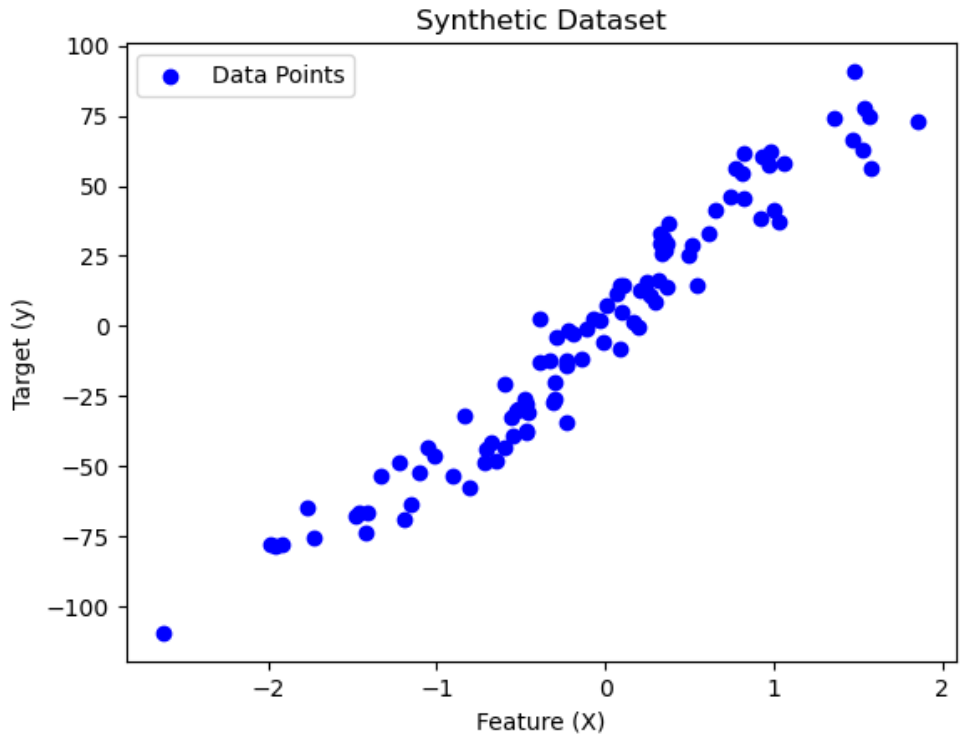
5. Repeat

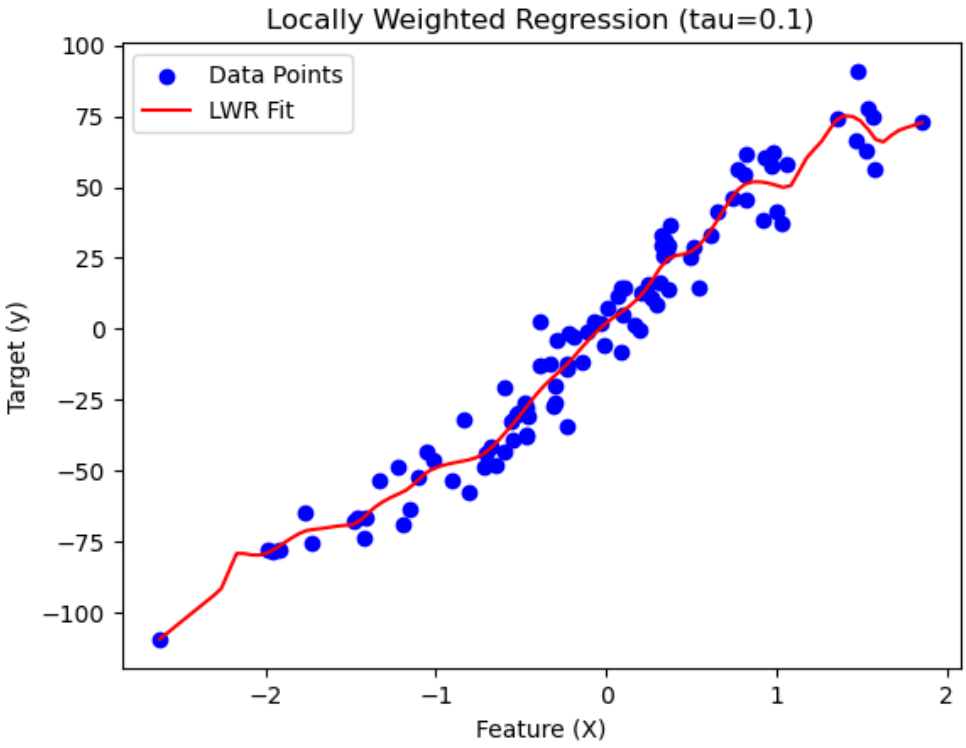
- Repeat steps 2-4 for all query points to obtain a smooth regression curve.

5.3 Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_regression
from sklearn.metrics import mean_squared_error

# Generate a synthetic dataset
np.random.seed(42)
X, y = make_regression(n_samples=100, n_features=1, noise=10, random_state=42)

# Add some non-linearity to the dataset
y = y + 10 * np.sin(X[:, 0] * 2)

# Plot the dataset
plt.scatter(X, y, color='blue', label='Data Points')
plt.title("Synthetic Dataset")
plt.xlabel("Feature (X)")
plt.ylabel("Target (y)")
plt.legend()
plt.show()
```

```
# Locally Weighted Regression function
def locally_weighted_regression(X, y, query_point, tau=0.1):
    # Compute weights using a Gaussian kernel
    weights = np.exp(-np.sum((X - query_point) ** 2, axis=1) / (2 * tau ** 2))

    # Add a bias term to X
    X_bias = np.c_[np.ones(X.shape[0]), X]

    # Compute the weighted least squares solution
    W = np.diag(weights)
    theta = np.linalg.inv(X_bias.T @ W @ X_bias) @ (X_bias.T @ W @ y)

    # Predict the target value for the query point
    query_point_bias = np.array([1, query_point[0]])
    y_pred = query_point_bias @ theta

    return y_pred

# Predict using Locally Weighted Regression
def predict_lwr(X_train, y_train, X_test, tau=0.1):
    y_pred = np.zeros(X_test.shape[0])
    for i, query_point in enumerate(X_test):
        y_pred[i] = locally_weighted_regression(X_train, y_train, query_point, tau)
    return y_pred

# Generate test points
X_test = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)

# Predict using LWR
tau = 0.1  # Bandwidth parameter
y_pred = predict_lwr(X, y, X_test, tau)

# Plot the results
plt.scatter(X, y, color='blue', label='Data Points')
plt.plot(X_test, y_pred, color='red', label='LWR Fit')
plt.title(f"Locally Weighted Regression (tau={tau})")
plt.xlabel("Feature (X)")
plt.ylabel("Target (y)")
plt.legend()
plt.show()

# Evaluate the model
mse = mean_squared_error(y, predict_lwr(X, y, X, tau))
print(f"Mean Squared Error (MSE) on Training Data: {mse:.4f}")
```






```
Mean Squared Error (MSE) on Training Data: 64.7316
```

Viva Questions

- What is Locally Weighted Regression (LWR)?
- How does LWR differ from ordinary least squares linear regression?
- What does the term ‘non-parametric’ mean in the context of LWR?
- Why do we need to assign weights to the data points in LWR?
- What is the role of the bandwidth parameter τ?
- What does the kernel function do in LWR?
- What happens if the value of τ is too small or too large?
- Why is LWR considered computationally expensive?
- What are the advantages and disadvantages of LWR?
- What are some real-world applications of LWR?